接口测试文档

一、简单测试用例

模块	接口名称	测试场景	输入数据	预期结果
认证	POST 登录	1. 正常登录	`{"user_id": "111", "password": "12345678", "verification_code_id": "abc123", "verification_code": "1234"}`	HTTP 200, 返回 `{"code": 200, "data": {"token": "..."}}`
		2. 密码错误	`{"user_id": "111", "password": "wrong", ...}`	HTTP 401, 返回 `{"code": 401, "msg": "密码错误"}`
	GET 获取图片验证码	1. 成功获取验证码	无参数	HTTP 200, 返回 `{"data": {"image": "base64字符串", "verification_code_id": "..."}}`
用户	GET 查询用户	1. 分页查询VIP用户	`?page=1&page_size=10&is_vip=true` (Header: `Authorization: Bearer xxx`)	HTTP 200, 返回分页数据且 `users[].is_vip` 均为 `true`
		2. 未授权访问	无 `Authorization` Header	HTTP 401
借书	POST 借书	1. 成功借书	`{"book_id": 1, "borrower_id": 111, "copy_id": 1, "expected_return_time": "2024-12-31"}`	HTTP 200, 返回 `{"data": {"book_id": 1, "copy_id": 1}}`

模块	接口名称	测试场景	输入数据	预期结果
		2. 借阅已借出的书籍	`{"book_id": 1, "copy_id": 1, ...}` (该副本已借出)	HTTP 400, 返回 `{"msg": "书籍不可借"}`
库存	POST 添加库存	1. 添加新书	`{"author": "鲁迅", "name": "呐喊", "category": "文学", "publisher": "人民文学", "quantity_added": 10}`	HTTP 200, 返回 `{"data": {"book_id": 新ID}}`
		2. 必填字段缺失	`{"name": "呐喊"}` (缺少 author/category)	HTTP 400, 返回 `{"msg": "缺少必填字段"}`
活动管理	POST 新增活动	1. 创建有效活动	`{"info": {"activity_name": "读书会", "start_time": "2024-10-01", "end_time": "2024-10-02", ...}}`	HTTP 200, 返回 `{"data": {"activity_id": 1}}`
		2. 结束时间早于开始时间	`{"info": {"start_time": "2024-10-02", "end_time": "2024-10-01", ...}}`	HTTP 400, 返回 `{"msg": "时间无效"}`
志愿者管理	POST 创建志愿者	1. 正常创建	`{"name": "张三", "phone": "13800138000", "age": 25, "expertiseArea": "教育"}`	HTTP 200, 返回 `{"data": {"volunteer_id": 1}}`

模块	接口名称	测试场景	输入数据	预期结果
		2. 手机号格式错误	`{"phone": "123", ...}`	HTTP 400, 返回 `{"msg": "手机号无效"}`
首页	GET 获取首页统计数据	1. 验证数据完整性	Header: `Authorization: Bearer xxx`	HTTP 200, 返回 `{"total_stock": 100, "today_borrowed": 5, ...}`

二、响应速度要求

接口分类	主要接口路径	最大响应时间（90%请求）	最大响应时间（极端）
登录相关	/api/v1/auth/login, /api/v1/auth/get_verification_code	≤500ms	≤1000ms
用户相关	/api/v1/user/**	≤800ms	≤1500ms
借书、库存相关	/api/v1/book/**	≤1000ms	≤2000ms
活动、捐赠、志愿者管理	/api/v1/activity/**, /api/v1/book/donate/**, /api/v1/volunteer/**	≤1200ms	≤2000ms

说明：

- 90%的接口响应应满足“最大响应时间”以内；
- 极端情况下（如服务器高峰期）允许单次请求稍超，但需在监控中及时告警。

三、部署方式

使用dockerfile打包，并使用docker-compose进行一键部署，并使用CI/CD实现自动化部署

本项目采用Docker技术进行容器化部署。首先，通过编写Dockerfile对应用进行打包，确保应用及其依赖环境一致，提升了部署的稳定性与可移植性。随后，使用docker-compose对多个服务进行统一管理，实现一键部署，简化了复杂应用的启动流程，极大提高了部署效率。

为了进一步优化部署流程，本项目还集成了CI/CD（持续集成/持续交付）机制。当代码推送到版本控制系统后，CI/CD流水线会自动完成代码构建、镜像生成、推送至镜像仓库，并触发远程服务器的自动部署，实现了从代码提交到上线的全流程自动化，减少了人为操作带来的失误，同时也加快了开发迭代速度。

通过以上部署方式，系统具备了快速部署、版本可控、回滚便捷等优点，为项目的稳定运行和后续扩展提供了坚实保障。